



Dhaka International University

Lab Report details:

Lab Report Topic : **Write a code for Round Robin Scheduling Algorithm using C/C++**

Lab Report No : 03

Course Title : Operating System

Course Code : 0613-301

Submitted By:	Submitted To:
S M Nabil Ausaf Roll: 01 Reg No : CS-D-91-23-124492	Md. Nazmus Sakib, Lecturer of CSE Department

Date of Submission:15.11.25

Operating System Lab Report

Experiment No: 04

Experiment Name: Implementation of Round Robin Scheduling Algorithm with Arrival Time

Objectives

To demonstrate understanding and implementation skills of the CPU Round Robin Algorithm using arrival times.

1. To find Waiting Time (WT) and Turnaround Time (TAT) for each process.
2. To determine the average waiting time and average turnaround time.
3. To explore how a time quantum impacts the efficiency of process scheduling.

Theory

Round Robin (RR) - Preemptive CPU Scheduling Algorithm.

Every process gets allotted a fixed time slot (known as a 'quantum'), to run in a cyclical fashion. In case a process does not complete execution within its quantum, it will move to the end of the queue. The CPU now turns to the other ready processes. It serves them in equal manner.

Key Terms:

Arrival Time (AT): Time when a process enters the ready queue.

Burst Time (BT): The total execution time for a process.

Waiting Time (WT): The period a process spends waiting in the ready queue.

Here, **WT = TAT - BT**

Turnaround Time (TAT): Turnaround Time (TAT), Activity Cycle Time (ACT), or Average

TAT = Completion Time - Arrival Time

Algorithm

1. Start
2. Input number of processes n.
3. Input **Arrival Time** and **Burst Time** for each process.
4. Input **Time Quantum**.
5. Initialize remaining burst time ($rem_bt[i] = bt[i]$).
6. Set current time $t = 0$.
7. While all processes are not completed:

Check each process:

If process has arrived ($at[i] \leq t$) and $rem_bt[i] > 0$:

If $rem_bt[i] > qt$, execute for qt and reduce remaining time.

Else, finish execution and calculate TAT and WT.

If no process is ready, increase t by 1.

8. After all processes complete, calculate **Average Waiting Time** and **Average Turnaround Time**.

9. Display results.

10. Stop.

CODE (WITH C)

```
Start here X lab3 witharrival time.c X
1  #include <stdio.h>
2
3  int main() {
4      int n, i, qt, count = 0, sq = 0, t = 0;
5      int bt[20], at[20], wt[20], tat[20], rem_bt[20];
6      float awt = 0, att = 0;
7
8      printf("Enter number of processes: ");
9      scanf("%d", &n);
10
11     // Input Arrival Time and Burst Time for each process
12     printf("Enter Arrival Time and Burst Time for each process:\n");
13     for (i = 0; i < n; i++) {
14         printf("P[%d] Arrival Time: ", i + 1);
15         scanf("%d", &at[i]);
16         printf("P[%d] Burst Time: ", i + 1);
17         scanf("%d", &bt[i]);
18         rem_bt[i] = bt[i];
19     }
20
21     printf("Enter Quantum Time: ");
22     scanf("%d", &qt);
23
24     int completed = 0;
25     int flag = 0;
26     while (completed < n) {
27         flag = 0;
28         for (i = 0; i < n; i++) {
29             if (rem_bt[i] > 0 && at[i] <= t) {
30                 flag = 1;
31                 if (rem_bt[i] > qt) {
32                     t += qt;
```

```

32         t += qt;
33         rem_bt[i] -= qt;
34     } else {
35         t += rem_bt[i];
36         tat[i] = t - at[i];
37         wt[i] = tat[i] - bt[i];
38         rem_bt[i] = 0;
39         completed++;
40     }
41 }
42 }
43 if (flag == 0) // No process has arrived yet
44     t++;
45 }
46
47 printf("\n\nProcess\tArrival\tBurst\tWaiting\tTurnaround\n");
48 for (i = 0; i < n; i++) {
49     awt += wt[i];
50     att += tat[i];
51     printf("P[%d]\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], wt[i], tat[i]);
52 }
53
54 awt /= n;
55 att /= n;
56
57 printf("\nAverage Waiting Time = %.2f", awt);
58 printf("\nAverage Turnaround Time = %.2f\n", att);
59
60 return 0;
61 }
62

```

OUTPUT

```

Enter number of processes: 3
Enter Arrival Time and Burst Time for each process:
P[1] Arrival Time: 1
P[1] Burst Time: 2
P[2] Arrival Time: 1
P[2] Burst Time: 3
P[3] Arrival Time: 2
P[3] Burst Time: 4
Enter Quantum Time: 2

```

Process	Arrival	Burst	Waiting	Turnaround
P[1]	1	2	0	2
P[2]	1	3	4	7
P[3]	2	4	4	8

```

Average Waiting Time = 2.67
Average Turnaround Time = 5.67

```

Result and Discussion

The system has successfully incorporated both Round-Robin and Arrival Time algorithms. Every process gets an equal chance to run in fixed time intervals. Both average waiting time and average turnaround time were computed correctly. The change in the quantum value directly impacts CPU efficiency. A smaller value increases context switching, whereas a larger value tends to resemble FCFS.

Conclusion

From this experiment, we conclude that:

- ❖ It supports fair processing. It's a time-sharing technique.
- ❖ It guarantees that every process receives access to CPU execution.
- ❖ Taking into account the arrival time makes it useful for real-time systems.